# Chapter 1:
# An Introduction to Computer Science

BMM 105 Introduction to Computer Engineering
Fall 2023

Assoc. Prof. Dr. Ayça Kolukısa

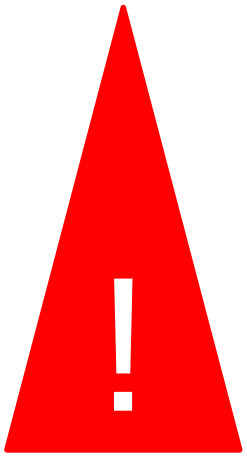
atarhan@cs.hacettepe.edu.tr

# Objectives

In this chapter, you will learn about

- The definition of computer science
- Algorithms
- A brief history of computing

# Introduction




- Common **misconceptions** about computer science
  - Computer science is the study of computers
  - Computer science is the study of how to write computer programs
  - Computer science is the study of the uses and applications of computers and software

# The Definition of Computer Science

- Gibbs and Tucker definition of computer science
  - **The study of algorithms:**
    - Formal and mathematical properties
    - Hardware realizations
    - Linguistic realizations
    - Applications
- Computer scientist designs and develops algorithms to solve problems

# Definition of Computer Science (cont'd)

- **Operations involved in designing algorithms**
  - Formal and mathematical properties
    - Studying the behavior of algorithms to determine whether they are correct and efficient
  - Hardware realizations
    - Designing and building computer systems that are able to execute algorithms

# Definition of Computer Science (cont'd)

- Operations involved in designing algorithms (cont'd)
  - Linguistic realizations
    - Designing programming languages and translating algorithms into these languages
  - Applications
    - Identifying important problems and designing correct and efficient software packages to solve these problems

# Definition of Computer Science (cont'd)

- **Algorithm**
  - Dictionary definition
    - Procedure for solving a mathematical problem in a finite number of steps that frequently involves repetition of an operation
    - A step-by-step method for accomplishing a task
  - Informal description
    - An ordered sequence of instructions that is guaranteed to solve a specific problem

# Definition of Computer Science (cont'd)

- An algorithm is a list that looks like
  - STEP 1: Do something.
  - STEP 2: Do something.
  - STEP 3: Do something.
  - . .
  - . .
  - . .
  - STEP N: Stop. You are finished.

# Video:

## Long way to a well-instructed algorithm!

- https://www.youtube.com/watch?v=Ct-lOOUqmyY&t=33s

- What problem(s) do you see here?

# Definition of Computer Science (cont'd)

- Categories of operations used to construct algorithms
  - **Sequential operations**
    - Carry out a single well-defined task; when that task is finished, the algorithm moves on to the next operation
    - Examples:
      - Add 1 cup of butter to the mixture in the bowl
      - Subtract the amount of the check from the current account balance
      - Set the value of x to 1

# Definition of Computer Science (cont'd)

- Categories of operations used to construct algorithms (cont'd)
  - **Conditional operations**
    - Ask a question and then select the next operation to be executed on the basis of the answer to that question
    - Examples:
      - If the mixture is too dry, then add one-half cup of water to the bowl

# Definition of Computer Science (cont'd)

- Conditional operations examples (continued):
  - If the amount of the check is less than or equal to the current account balance, then cash the check; otherwise, tell the person that the account is overdrawn
  - If x is not equal to 0, then set y equal to 1/x; otherwise, print an error message that says we cannot divide by 0

# Definition of Computer Science (cont'd)

- Categories of operations used to construct algorithms (cont'd)
  - **Iterative operations**
    - Tell us to go back and repeat the execution of a previous block of instructions
    - Examples:
      - Repeat the previous two operations until the mixture has thickened
      - While there are still more checks to be processed, do the following five steps
      - Repeat steps 1, 2, and 3 until the value of y is equal to 11

# Algorithm for Shampooing Your Hair

**FIGURE 1.3(a)**

*A Correct Solution to the Shampooing Problem*

**Algorithm for Shampooing Your Hair**

| STEP | OPERATION |
|---|---|
| 1 | Wet your hair |
| 2 | Set the value of *WashCount* to 0 |
| 3 | Repeat steps 4 through 6 until the value of *WashCount* equals 2 |
| 4 | Lather your hair |
| 5 | Rinse your hair |
| 6 | Add 1 to the value of *WashCount* |
| 7 | Stop, you have finished shampooing your hair |

# Another way (without iteration condition)

**FIGURE 1.3(b)**

*Another Correct Solution to the Shampooing Problem*

**Another Algorithm for Shampooing Your Hair**

| STEP | OPERATION |
|---|---|
| 1 | Wet your hair |
| 2 | Lather your hair |
| 3 | Rinse your hair |
| 4 | Lather your hair |
| 5 | Rinse your hair |
| 6 | Stop, you have finished shampooing your hair |

# Definition of Computer Science (cont'd)

- If we can specify an algorithm to solve a problem, we can automate its solution
- **Computing agent:**
  - The machine, robot, person, or thing carrying out the steps of the algorithm
  - Does not need to understand the concepts or ideas underlying the solution

# Formal Definition of an Algorithm (cont'd)

- A primitive operation (or a primitive) of the computing agent
  - Operation that is unambiguous for computing agent
  - Primitive operations of different individuals (or machines) vary
  - An algorithm must be composed entirely of primitives
- Effectively computable
  - Computational process exists that allows computing agent to complete that operation successfully

# Formal Definition of an Algorithm (cont'd)

- Step 1 is not effectively computable

| | |
|---|---|
| STEP 1: | Generate a list $L$ of all the prime numbers: $L_1, L_2, L_3, \ldots$ |
| STEP 2: | Sort the list $L$ in ascending order |
| STEP 3: | Print out the 100th element in the list, $L_{100}$ |
| STEP 4: | Stop |

- Why we had to initialize the value of the variable?

Write out the exact decimal value of $\pi$. ($\pi$ cannot be represented exactly.)
Set *average* to (*sum* ÷ *number*). (If *number* = 0, division is undefined.)
Set the value of *result* to $\sqrt{N}$. (If $N < 0$, then *result* is undefined if you are using real numbers.)
Add 1 to the current value of *x*. (What if *x* currently has no value?)

# A BRIEF HISTORY OF COMPUTING

# The Early Period: Up to 1940

- 3,000 years ago: Mathematics, logic, and numerical computation
  - Important contributions made by the Greeks, Egyptians, Babylonians, Indians, Chinese, Persians
- 1614: Logarithms
  - Invented by John Napier to simplify difficult mathematical computations
- Around 1622: First slide rule created

# The Early Period: Up to 1940 (cont'd)

- 1672: The Pascaline
  - Designed and built by Blaise Pascal
  - One of the first mechanical calculators
  - Could do addition and subtraction
- 1674: Leibnitz's Wheel
  - Constructed by Gottfried Leibnitz
  - Mechanical calculator
  - Could do addition, subtraction, multiplication, and division

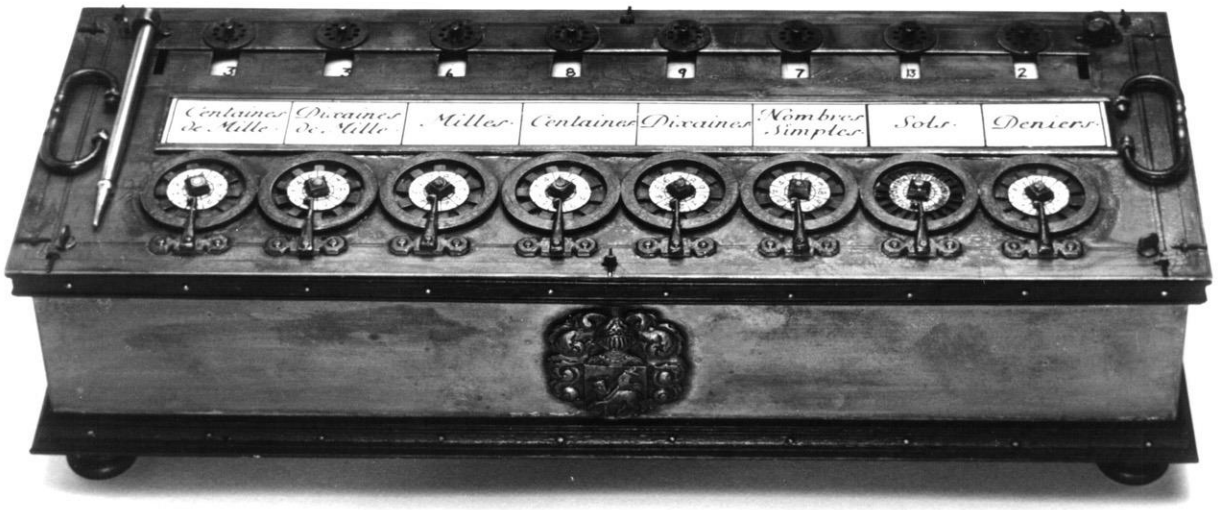
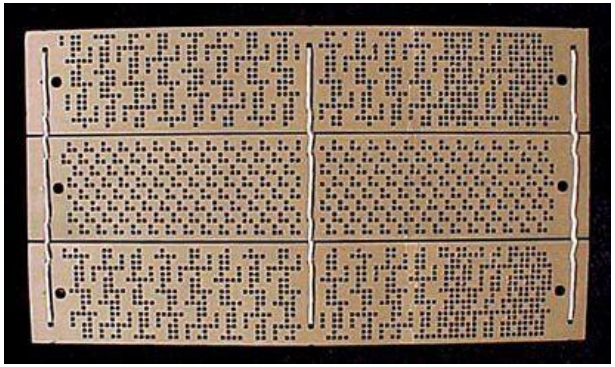



Figure 1.4
The Pascaline: One of the Earliest Mechanical Calculators

# The Early Period: Up to 1940 (cont'd)



- 1801: The Jacquard Loom
  - Developed by Joseph Jacquard
  - Automated loom
  - Used punched cards to create desired pattern
- 1823: The Difference Engine
  - Developed by Charles Babbage
  - Did addition, subtraction, multiplication, and division to 6 significant digits
  - Solved polynomial equations and other complex mathematical problems

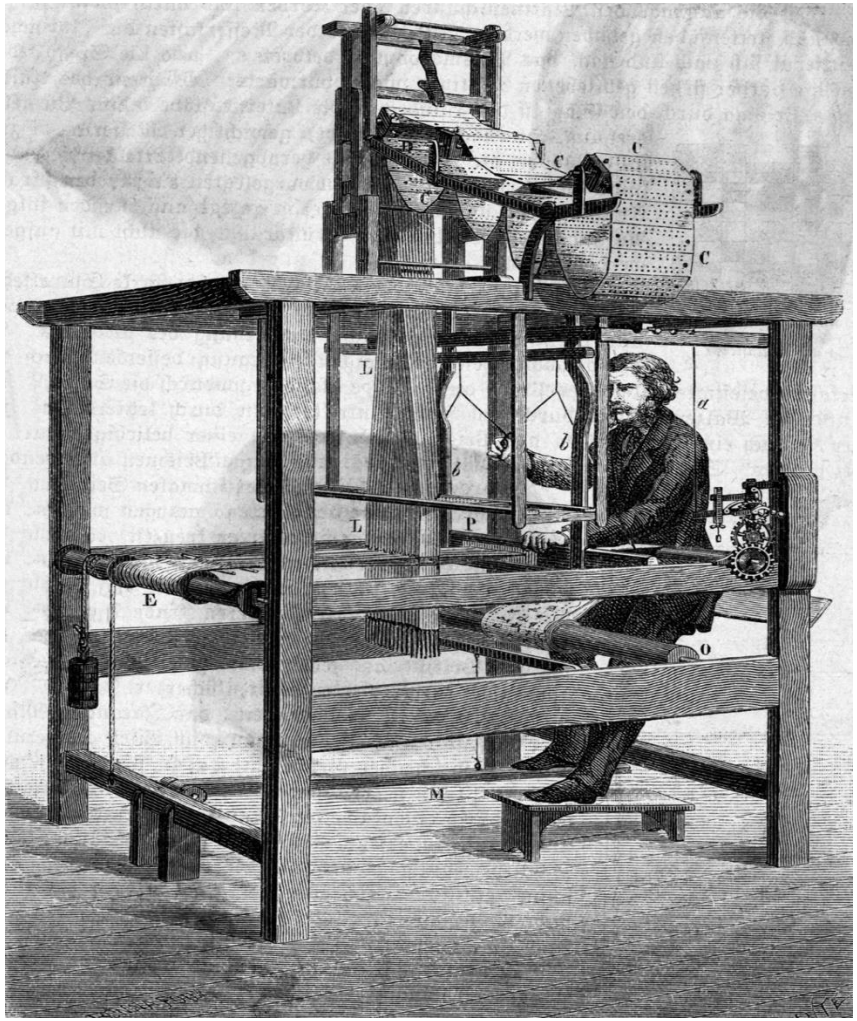



Figure 1.5
Drawing of the Jacquard Loom

# The Early Period: Up to 1940 (cont'd)

- 1830s: The Analytic Engine
  - Designed by Charles Babbage
  - More powerful and general-purpose computational machine
  - Components were functionally similar to the four major components of today's computers
    - Mill (modern terminology: arithmetic/logic unit)
    - Store (modern terminology: memory)
    - Operator (modern terminology: processor)
    - Output (modern terminology: input/output)

# The Early Period: Up to 1940 (cont'd)

- 1890: U.S. census carried out with programmable card processing machines
  - Built by Herman Hollerith
  - These machines could automatically read, calculate, and sort data entered on punched cards

# The Birth of Computers: 1940-1950

- Development of electronic, general-purpose computers
  - Did not begin until after 1940
  - Was fueled in large part by needs of World War II
- Early computers
  - Mark I
  - ENIAC (Electronic Numerical Integrator And Computer)
  - ABC system
  - Colossus
  - Z1

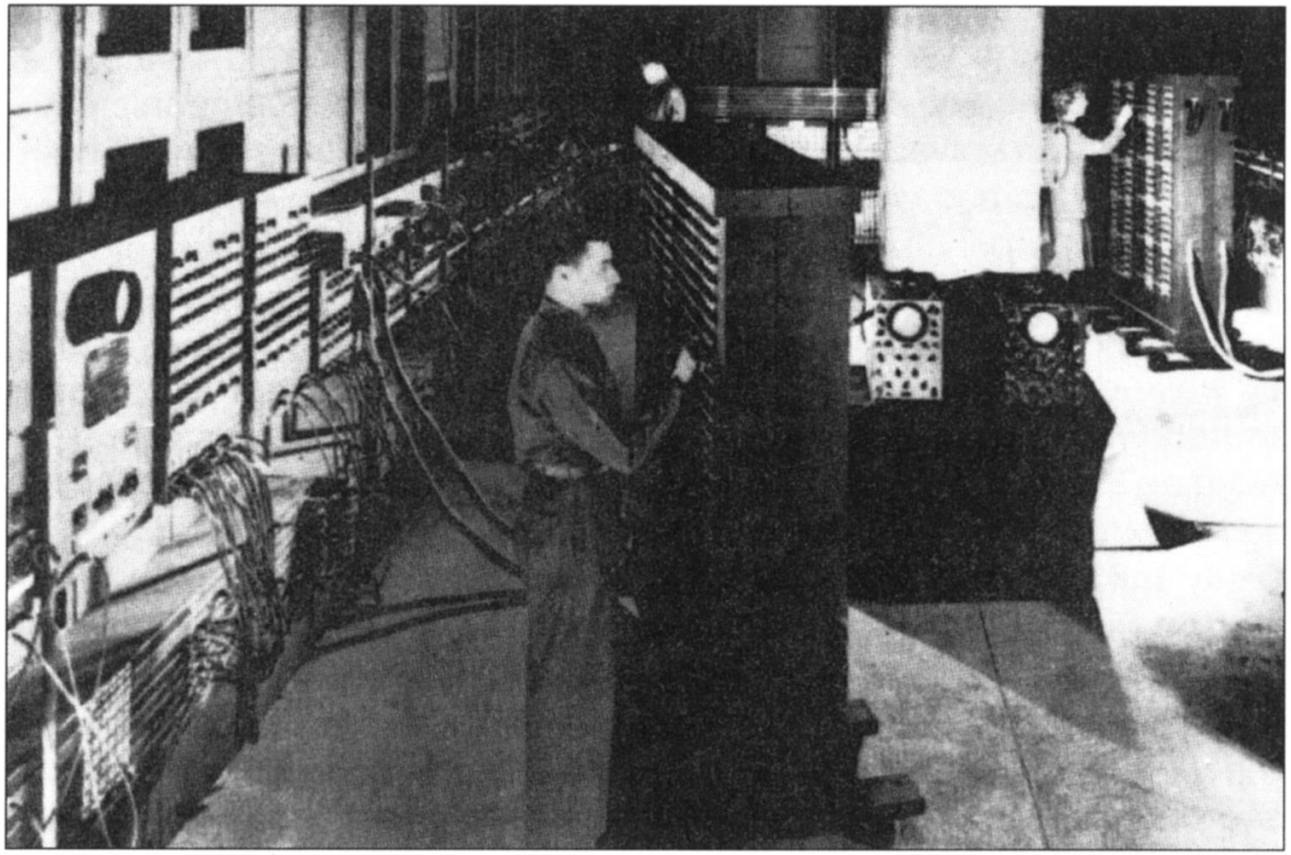
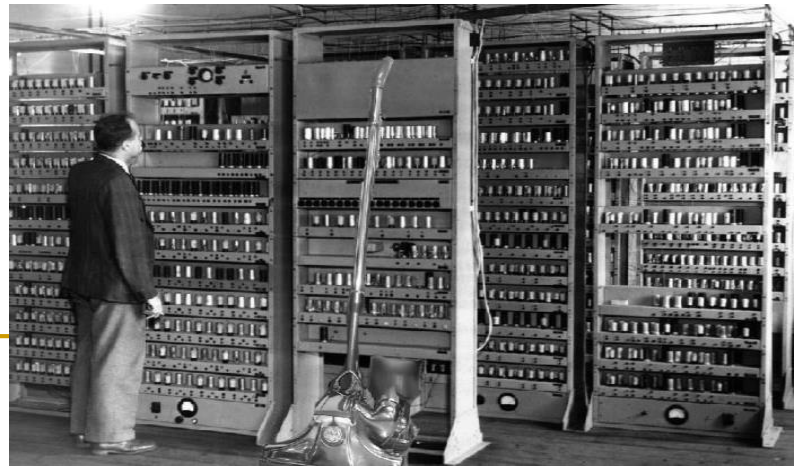


Figure 1.6
Photograph of the ENIAC Computer

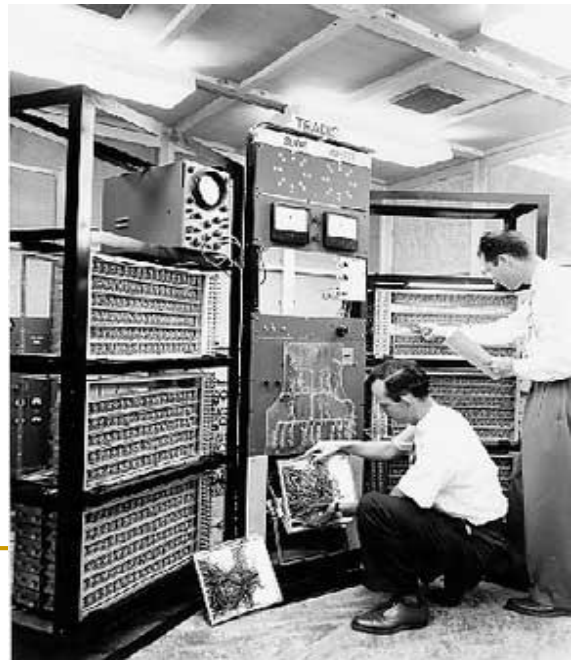
# Birth of Computers: 1940-1950 (cont'd)

- Stored program computer model
  - Proposed by John Von Neumann in 1946
  - Stored binary algorithm in the computer's memory along with the data
  - Is known as the Von Neumann architecture
  - Modern computers remain, fundamentally, Von Neumann machines
  - First stored program computers
    - EDVAC
    - EDSAC



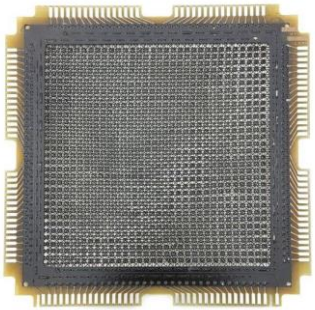
# The Modern Era: 1950 to the Present

- First generation of computing (1950-1957)
  - Vacuum tubes used to store data and programs
  - Each computer was multiple rooms in size
  - Computers were not very reliable



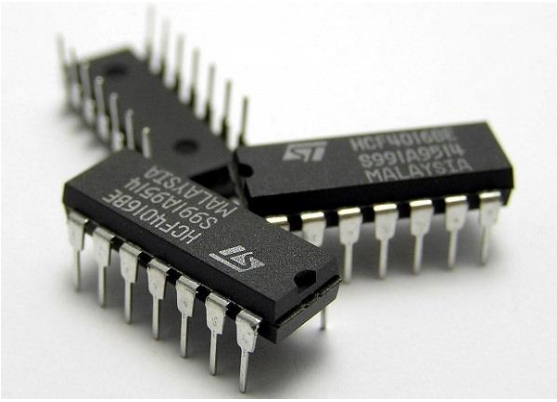
# Modern Era: 1950 to the Present (cont'd)

- Second generation of computing (1957-1965)
  - Transistors and magnetic cores replaced vacuum tubes
  - Dramatic reduction in size
    - Computer could fit into a single room
  - Increase in reliability of computers
  - Reduced cost of computers
  - High-level programming languages
    - The programmer occupation was born



# Modern Era: 1950 to the Present (cont'd)

- Third generation of computing (1965-1975)
  - Integrated circuits rather than individual electronic components were used
  - Further reduction in size and cost of computers
    - Computers became desk-sized
    - First minicomputer developed
  - Software industry formed



# Modern Era: 1950 to the Present (cont'd)

- Fourth generation of computing (1975-1985)
  - Reduced to the size of a typewriter
  - First microcomputer developed
  - Desktop and personal computers common
  - Appearance of
    - Computer networks
    - Electronic mail
    - User-friendly systems (graphical user interfaces)
    - Embedded systems

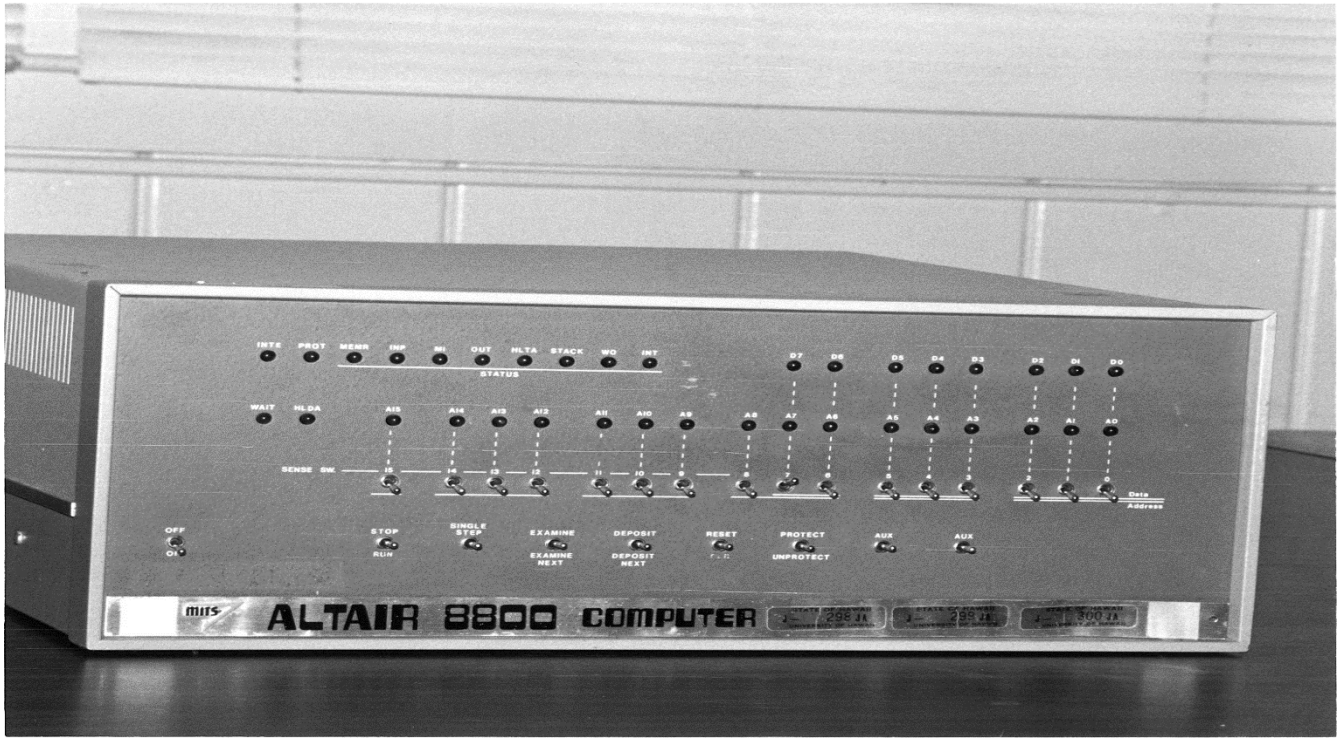



Figure 1.7
The Altair 8800, the World's First Microcomputer

# Modern Era: 1950 to the Present (cont'd)

- Fifth generation of computing (1985-?)
  - Recent developments
    - Massively parallel processors
    - Handheld devices and other types of personal digital assistants (smart phones)
    - High-resolution graphics
    - Powerful multimedia user interfaces incorporating sound, voice recognition, touch, photography, video, and television

# Modern Era: 1950 to the Present (cont'd)

- Recent developments (cont'd)
  - Integrated global telecommunications incorporating data, television, telephone, fax, the Internet, and the World Wide Web
  - Wireless data communications
  - Massive storage devices
  - Ubiquitous computing

| Generation | Approximate Dates | Major Advances |
|---|---|---|
| First | 1950–1957 | First commercial computers<br>First symbolic programming languages<br>Use of binary arithmetic, vacuum tubes for storage<br>Punched card input/output |
| Second | 1957–1965 | Transistors and core memories<br>First disks for mass storage<br>Size reduction, increased reliability, lower costs<br>First high-level programming languages<br>First operating systems |
| Third | 1965–1975 | Integrated circuits<br>Further reduction in size and cost, increased reliability<br>First minicomputers<br>Time-shared operating systems<br>Appearance of the software industry<br>First set of computing standards for compatibility between systems |

Figure 1.8. Some of the Major Advancements in Computing

| | | |
|---|---|---|
| Fourth | 1975–1985 | Large-scale and very-large-scale integrated circuits<br>Further reduction in size and cost, increased reliability<br>First microcomputers<br>Growth of new types of software and of the software industry<br>Computer networks<br>Graphical user interfaces |
| Fifth | 1985–? | Ultra-large-scale integrated circuits<br>Supercomputers and parallel processors<br>Laptops and handheld computers<br>Wireless computing<br>Massive external data storage devices<br>Ubiquitous computing<br>High-resolution graphics, visualization, virtual reality<br>Worldwide networks<br>Multimedia user interfaces |

Figure 1.8. Some of the Major Advancements in Computing

# Summary

- Computer science is the study of algorithms
- An algorithm is a well-ordered collection of unambiguous and effectively computable operations that, when executed, produces a result and halts in a finite amount of time
- If we can specify an algorithm to solve a problem, then we can automate its solution
- Computers developed from mechanical calculating devices to modern electronic marvels of miniaturization